



WEEK 02

Management of Information Systems (IS) Assessment

The Four Machines Behind Every Business Decision

Two hours · Four lessons · Four working code examples · Designed for asynchronous, offline study

Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together!

How to use this deck

- This deck is the **spine**. The lecture document is the **script**. Every slide maps to a numbered section you can read in full offline.
- **For the students:** you can complete this week entirely offline. Only the four videos need a connection, and each has a written alternative.
- **Four downloadable code files, six diagrams, four spreadsheets and seven reference tables** accompany this deck. Nothing requires installation beyond Python.
- Every abbreviation is expanded in round brackets the first time it appears — on the slide and in the lecture.



Learning outcomes — eight things you will be able to do

- **Name and place** the seven components of an IT (Information Technology) infrastructure, and say what breaks if each is missing.
- **Distinguish** IaaS (Infrastructure as a Service), PaaS (Platform as a Service) and SaaS (Software as a Service) by one question: who is responsible for what?
- **Calculate** a five-year TCO (Total Cost of Ownership) comparison between owning hardware and renting it, and state the break-even point.
- **Convert** a disorganised spreadsheet into a normalised relational database with primary keys and foreign keys, and explain what each key prevents.
- **Write and read** a SQL (Structured Query Language) query that joins two tables and aggregates a result — the raw material of business intelligence.
- **Trace** a single web request through the four layers of the TCP/IP (Transmission Control Protocol / Internet Protocol) model, naming what each layer adds.
- **Diagnose** a slow Wi-Fi (Wireless Fidelity) problem using a channel plan and a bandwidth budget, rather than guessing.
- **Quantify** a security risk using SLE (Single Loss Expectancy), ARO (Annualized Rate of Occurrence) and ALE (Annualized Loss Expectancy) — and use the result to justify, or refuse, a security purchase.

Nothing here says “list the advantages of X.” Every outcome is a **calculation, a trace, a conversion or a diagnosis** — something you can be wrong about in a way you can check.

The vocabulary you need before we start

Abbreviation	Expansion	What it means in one line
IT	Information Technology	All hardware, software and services that handle data
IS	Information System	Technology plus the people and processes around it
TCO	Total Cost of Ownership	Every cost over the life of a decision, not the sticker price
SQL	Structured Query Language	The language used to ask a database questions
DBMS	Database Management System	The software that stores and protects the data
TCP/IP	Transmission Control Protocol / Internet Protocol	The four-layer rulebook the internet runs on
ALE	Annualized Loss Expectancy	Expected cost of a risk per year, in currency
ROSI	Return on Security Investment	Whether a control is worth more than it costs

The full table has **35 rows** and lives in Section 2 of the lecture, plus a downloadable CSV (Comma-Separated Values) file with 54 entries.

LESSON 1

IT Infrastructure and Emerging Technologies

Sections 1.1 – 1.9 · ⌚ 00:12 → 00:42

What infrastructure is, and four eras of it

- Infrastructure is the **shared set of resources** every application depends on but none owns. The test: if it disappeared, would **more than one** system stop working?
 - If yes, it is infrastructure — budgeted, replaced and secured centrally.

Era	Roughly	Where the computing lived	Who controlled it
Mainframe	1959–1980s	One large central machine	A central IT department
Personal computer	1981–1990s	On every desk, separately	The individual user
Client/server	1983–present	Split between desktop and server	Shared: IT owns the server
Cloud & mobile	2000–present	Someone else's data centre	Rented, governed by contract

The pattern is **not** “newer is better” — it is control moving between the centre and the edge. Cloud is the mainframe idea again, but the centre is a company you do not own.

The seven components of an IT infrastructure

#	Component	What it provides	What breaks without it
1	Computing platforms	Processing capacity: servers, desktops, phones	Nothing runs at all
2	Operating system platforms	The layer software is written against	Applications have no host
3	Enterprise applications	Shared business functions (ERP, CRM)	Departments re-invent processes
4	Data management & storage	Where facts are kept and protected	Data lost, duplicated, contradictory
5	Networking & telecommunications	Movement of data between everything else	Isolated machines; nothing shares
6	Internet platforms	Public reach: web servers, DNS, hosting	No customer-facing presence
7	Consulting & system integration	The work of making 1–6 function together	Parts exist but do not cooperate

Component 7 is the one students forget and organisations pay for. Buying the parts is not the same as owning a working system. ERP (Enterprise Resource Planning) · CRM (Customer Relationship Management) · DNS (Domain Name System).

Virtualization: the idea that made the cloud possible

- Virtualization presents **one physical machine as several independent machines**, each with its own operating system.
- The economic problem it solved: a dedicated server typically ran at **10–15% utilisation**. The other 85% was purchased and wasted.
 - Consolidating ten such servers onto one physical host recovers that waste — less hardware, less electricity, less cooling, less rack space.
- It also makes a server a **file**. A file can be copied, moved, backed up and restarted on different hardware in minutes.
 - This is why cloud providers can allocate you a server in seconds: they are not installing anything, they are starting a file.

The cost consequence: capacity stops being something you buy in advance and becomes something you switch on. That is what the TCO model in Section 1.7 measures.

Three service models, one question

- The only question that separates them: **who is responsible for what?**
- **IaaS** (Infrastructure as a Service) — you rent the machine. You install and patch the operating system, the runtime and the application.
- **PaaS** (Platform as a Service) — you rent a place to run code. The provider patches the operating system and runtime; you own the code and the data.
- **SaaS** (Software as a Service) — you rent finished software. You configure it, you supply the data, and that is all you control.
- Moving down the list buys convenience and sells control. Neither is free.

Who is responsible for what

Layer	On-premises	IaaS	PaaS	SaaS
Applications	You	You	You	Provider
Runtime / middleware	You	You	Provider	Provider
Operating system	You	You	Provider	Provider
Virtualization	You	Provider	Provider	Provider
Servers & storage	You	Provider	Provider	Provider
Networking & facility	You	Provider	Provider	Provider
Your data	You	You	You	You

Read the last row again. The data row never moves. “We moved to SaaS so data protection is the provider's problem” is the single most expensive sentence in this lecture.

Emerging technologies, judged honestly

- Three questions to ask **before** adopting anything described as emerging:
 - **1. What problem does this solve that I already have?** If you cannot name the existing problem, you are buying a solution looking for one.
 - **2. What does it cost to leave?** Data export formats, retraining, contract exit terms. This question uncovers lock-in.
 - **3. Who is accountable when it is wrong?** Especially for anything that produces a recommendation rather than a fact.
- Applied to edge computing, quantum computing, generative AI (Artificial Intelligence) and IoT (Internet of Things), these three questions filter most vendor claims in about five minutes.

Question 2 is the one that uncovers **lock-in** — and the one vendors are least prepared to answer in writing.

Real-life example: the Print Hub has to decide

- **The task:** decide whether the Northside Campus Print Hub should buy a server outright or rent equivalent capacity, and defend the answer with a number.
- **How it relates:** this is Outcome 3. It converts the abstract IaaS/on-premises responsibility split into a five-year cash figure a manager can act on.
- Buy: CAD 4,200 server + 800 rack/UPS + 600 licences, then 430 electricity, 520 support and 6 admin hours per week each year, plus a 1,900 refresh in year 4.
- Rent: CAD 900 migration, then 1,560 virtual machine, 340 managed backup, 290 network egress and 2 admin hours per week each year.
- Administrator time is valued at **CAD 18/hour over 34 working weeks**.

All figures are **fictional**, chosen to be realistic. UPS (Uninterruptible Power Supply). CAD (Canadian Dollar).

The five-year model decides it

Year	Buy — cumulative	Rent — cumulative	Difference	Cheaper
0	CAD 5,000	CAD 900	4,100	Rent
1	CAD 10,222	CAD 4,314	5,908	Rent
2	CAD 15,444	CAD 7,728	7,716	Rent
3	CAD 20,666	CAD 11,142	9,524	Rent
4	CAD 27,788	CAD 14,556	13,232	Rent
5	CAD 33,010	CAD 17,970	15,040	Rent

- **Break-even: year 0.** Renting is cheaper from the first day and the gap widens every year. There is no crossover to wait for.
- The year-4 jump in the buy column is the hardware refresh — the cost that never appears on the original quotation.

Sensitivity: the input everyone omits

Administrator hourly rate	Buy — 5-year	Rent — 5-year	Recommendation
CAD 0 (volunteer labour)	CAD 14,650	CAD 11,850	Rent
CAD 9	CAD 23,830	CAD 14,910	Rent
CAD 18 (baseline)	CAD 33,010	CAD 17,970	Rent
CAD 27	CAD 42,190	CAD 21,030	Rent
CAD 40 (professional rate)	CAD 55,450	CAD 25,450	Rent

Labour is the largest single variable and the one most often left out of the comparison. Even valued at zero, renting still wins here — but notice how much faster the buy column grows once time is priced at all.

Code Example 1 — TCO calculator (Python)

```
# Define a function that calculates the total cost incurred in ONE specific year.  
# Separating 'one year' from 'all years' keeps the logic testable and easy to read.  
def cost_in_year(option: CostOption, year: int, hourly_rate: float) -> float:  
    # Year 0 carries the up-front cost and nothing else: nothing has run yet.  
    if year == 0:  
        return option.upfront  
    # Every later year carries the recurring cost plus the value of staff time.  
    total = option.recurring  
    total += option.admin_hours_per_week * WEEKS_PER_YEAR * hourly_rate  
    # Add any one-off cost scheduled for this particular year (the refresh).  
    total += option.scheduled.get(year, 0.0)  
    return total
```

- **Task:** turn the Print Hub decision into a program that recomputes the answer when the assumptions change, so the recommendation survives new information.
- **Relation to the lecture:** this is Outcome 3 in executable form. The file is also the base for Section 1 of the graded lab task.
- Full file: **150 lines**, every line commented above itself — ``code/L1_tco_calculator.py``

Lesson 1 — video and checkpoint

- **Video (under 15 minutes):** “IaaS Explained” — IBM Technology, YouTube.
 - https://www.youtube.com/watch?v=XRdmfo4M_YA
 - Watch on the publisher's channel only. Do not download, re-upload or redistribute. The video is **linked, not embedded or copied** — see Section 12.4.

- **Checkpoint — answer before moving on (answers in Section 10):**
 - 1. “We moved to SaaS, so data protection is the provider's problem.” Using the responsibility table, say in one sentence why this is wrong.
 - 2. A server runs at 15% utilisation. In economic terms, what does virtualization change about that situation?
 - 3. In the Print Hub model, which single input has the largest effect — and why is it the one most often omitted?
 - 4. Name the three adoption questions, and say which one uncovers lock-in.

LESSON 2

Foundations of Business Intelligence: Databases and Information Management

Sections 2.1 – 2.10 · ⌚ 00:42 → 01:12

Five diseases of the file environment

Disease	What it means	The consequence you can see
Data redundancy	The same fact stored in several places	Storage waste, and no single source of truth
Data inconsistency	Those copies disagree	Two reports, two answers, both defensible
Program–data dependence	Code is tied to the file's exact layout	Adding one column breaks working programs
Poor security	Access is all-or-nothing per file	Anyone who can read anything can read everything
Lack of data sharing	Departments keep private copies	The organisation cannot answer its own questions

These five are not historical curiosities. **Every one of them is present in the Robotics Club spreadsheet** you will normalise in Section 2.8.

The relational model in one page

- A database is an **organised collection of data with rules the software enforces**. The rules are the entire point — without them it is a spreadsheet.
- **Table (relation)** — one subject, and only one. Members. Items. Loans.
- **Row (tuple)** — one instance of that subject. One member. One loan.
- **Column (attribute)** — one fact about that subject, with one data type.
- The governing discipline: **each fact is stored exactly once, in the table where it belongs**, and referenced everywhere else by a key.
 - If a person's email appears in forty loan rows, you have forty chances to be wrong and forty places to fix when they change it.

Keys: what each one prevents

Key	Definition	What it makes impossible
Primary key	A column whose value uniquely identifies each row and is never empty	Two rows for the same real thing; a row you cannot address
Foreign key	A column holding the primary key of another table	A loan pointing at a member who does not exist
Composite key	Two or more columns that are unique only together	Duplicate pairings, e.g. the same student enrolled twice in one course

- Keys are not bookkeeping. **A key is a rule the database will refuse to break**, including when the person breaking it is you, at 2 a.m., in a hurry.
- Code Example 2 proves this by deliberately trying to break both.

SQL: five statements do almost everything

```
SELECT name, email FROM members WHERE program = 'Engineering';  
    -- retrieve: choose columns, choose rows  
  
INSERT INTO items (name, category, cost) VALUES ('Oscilloscope B', 'Test equipment', 2900);  
    -- create: add a new row, subject to every constraint on the table  
  
UPDATE loans SET returned = 1 WHERE loan_id = 14;  
    -- modify: change existing rows that match the condition  
  
DELETE FROM loans WHERE loan_id = 14;  
    -- remove: delete matching rows (the WHERE clause is not optional in practice)  
  
SELECT category, COUNT(*) FROM items GROUP BY category ORDER BY COUNT(*) DESC;  
    -- aggregate: turn many rows into a ranked summary -- this is business intelligence
```

The last statement is the important one. JOIN and GROUP BY are where a database stops storing facts and starts answering questions.

From data to decision — and who governs it

- **The BI pipeline.** BI (Business Intelligence) is the whole chain, not the dashboard:
 - **1. Source systems** → **2. ETL** (Extract, Transform, Load) → **3. Data warehouse** → **4. Analysis** (queries, OLAP, models) → **5. Decision**
 - Step 5 is the only one that produces value. Everything above it is cost.
 - It almost always fails at step 2 — because the source data was never governed.
- **Governance answers four questions in writing, before the data exists:**
 - **Who owns it?** Not who stores it — who is accountable for it being correct.
 - **What is the definition?** Two departments, two definitions of “active student”, two incompatible reports.
 - **How long do we keep it?** A legal question as much as a technical one.
 - **Who may see it?** By role, reviewed, revoked when the role ends.

Data quality is measured on five dimensions: **accuracy, completeness, consistency, timeliness and validity**. OLAP (Online Analytical Processing).

Real-life example: the Robotics Club sheet

- **The task:** take one spreadsheet exhibiting all five diseases and convert it into three related tables that make the diseases structurally impossible.
- **How it relates:** this is Outcomes 4 and 5. It is the smallest complete example of the discipline every production database is built on.
- The original sheet holds six loan rows and contains: “Maria Silva”, “maria silva” and “M. Silva” — **one human being, three identities**; “Microcontroller” and “Microcontroler” as separate categories; and an email address retyped in every row.
- A COUNTIF over the name column reports **4 distinct people. The real answer is 3.** The sheet cannot tell, and never will.

The redesign, and what it now refuses

Table	Holds	Key	Now impossible
members	Each person exactly once: name, email, programme	member_id (PK)	Three spellings of one person
items	Each piece of equipment once: name, category, cost	item_id (PK)	“Microcontroler” — CHECK constraint rejects it
loans	Each borrowing event: who, what, out, due, returned	loan_id (PK), member_id + item_id (FK)	A loan for a member who does not exist

- PK (Primary Key) · FK (Foreign Key). The loans table stores **no names, no emails, no categories** — only keys pointing at the tables where those facts live once.
- Outstanding equipment value recovered by a three-table JOIN: **CAD 1,789.00** — a figure the original sheet could not produce at all.

Code Example 2 — build it, break it, query it

```
# Turn on foreign key enforcement. SQLite has foreign keys but leaves them OFF by
# default for historical compatibility. Without this line the database would silently
# accept a loan pointing at a member who does not exist -- exactly what we prove it stops.
connection.execute("PRAGMA foreign_keys = ON")

# Create the items table with a CHECK constraint restricting category to an approved
# list. This is the line that makes 'Microcontroler' impossible to store, ever.
cursor.execute("""
    CREATE TABLE items (
        item_id  INTEGER PRIMARY KEY,
        name     TEXT NOT NULL UNIQUE,
        category TEXT NOT NULL CHECK (category IN
            ('Microcontroller', 'Test equipment', 'Tools', 'Consumable')),
        replacement_cost REAL NOT NULL CHECK (replacement_cost >= 0))""")
```

- **Task:** build the three tables in real SQL, then deliberately attempt two illegal writes and show the database refusing both. **Relation:** Outcomes 4 and 5.
- Runs with no installation — SQLite ships inside Python, in memory, no file written. **225 lines** · `code/L2_club_inventory_db.py`

Lesson 2 — video and checkpoint

- **Video (under 15 minutes):** “What is a Relational Database?” — IBM Technology, YouTube.
 - <https://www.youtube.com/watch?v=OqjJppjDRLc>
 - Linked under the terms in Section 1.8 and Section 12.4: viewed on the publisher's own channel, never copied, never re-hosted.
- **Checkpoint:**
 - 1. A sheet lists the same person three ways. Which of the five diseases is that, and which key eliminates it?
 - 2. Explain what a CHECK constraint prevents that careful data entry does not.
 - 3. Why does the loans table contain no email addresses?
 - 4. Which SQL clause turns rows into a ranked summary, and why is that the moment data becomes business intelligence?

LESSON 3

Telecommunications, the Internet, and Wireless Technology

Sections 3.1 – 3.8 · 🕒 01:17 → 01:44

Four layers, and why layering is the invention

Layer	Responsibility	Adds to the data	Example
Application	What the data means	The request itself	HTTP, DNS, SMTP
Transport	Reliable delivery to the right program	Port numbers, sequence numbers	TCP, UDP
Internet	Getting between networks	Source and destination IP addresses	IP, ICMP
Network access	Getting across one physical link	MAC addresses, error checking	Ethernet, Wi-Fi

- Each layer talks only to the layers directly above and below it. **You can replace any one of them without touching the others** — which is why the internet survived the move from copper to fibre to radio.
- HTTP (HyperText Transfer Protocol) · SMTP (Simple Mail Transfer Protocol) · UDP (User Datagram Protocol) · MAC (Media Access Control)

One request, traced end to end

Step	What happens	If this fails, the symptom is...
1	Browser needs the IP address for the name typed	—
2	DNS query is sent and answered	“Server not found”, instantly
3	TCP connection opened to the server (three-way handshake)	Page hangs, then times out
4	TLS negotiated; certificate checked	Certificate warning; padlock absent
5	HTTP request sent	Blank page or protocol error
6	Server processes and responds	Slow only on this one site
7	Response travels back, hop by hop	Slow everywhere, all sites equally
8	Browser renders the page	Text appears, images do not

This table is a diagnostic tool, not trivia. The symptom tells you the step, and the step tells you the layer. TLS (Transport Layer Security).

Wireless is shared, and bandwidth is not latency

- A wired port gives each device its **own** capacity. An access point gives every nearby device a **share of one** capacity — only one may transmit at a time.
 - That waiting is **airtime contention**, and it is invisible on every speed test.
- In 2.4 GHz there are only **three non-overlapping channels: 1, 6 and 11**. Six access points all on channel 6 deliver roughly **one unit's worth**, not six.

Term	Definition	Unit	What suffers when it is bad
Bandwidth	How much data can move per second	Mbps	Large downloads, video quality
Latency	How long one packet takes to arrive	ms	Video calls, gaming, remote desktop
Jitter	Variation in that delay	ms	Voice and video call quality specifically
Packet loss	Percentage of packets that never arrive	%	Everything, unpredictably

Buying bandwidth cannot fix latency, and cannot fix airtime. These are independent capacities. Mbps (Megabits per second) · ms (milliseconds).

Real-life example: residence Wi-Fi dies at eight

- **Task:** decide whether Ashworth Residence should upgrade its internet connection, using measurement rather than the loudest complaint. **Relation:** Outcome 7.
- 180 residents · 500 Mbps link · six access points, **all on channel 6** · complaints nightly from 20:00 · proposed fix: 1 Gbps for **CAD 6,000/year**. All figures fictional.

Measure	Value	Reading
Active users at peak	99 of 180	55% concurrency
Total demand	306.9 Mbps	—
Internet link utilisation	61.4%	Headroom available — the link is NOT the constraint
Radios that add real capacity	1 of 6	Co-channel contention: the units share one channel
Effective wireless capacity	180 Mbps	Not 1,080 — six radios on one channel take turns
Wireless utilisation	170.5%	SATURATED — wireless airtime is the constraint
Clients per access point	45	Above the 30 guideline — contention expected

Verdict: the CAD 6,000 upgrade would have changed nothing. The link was at 61%. Re-planning the channels onto 1/6/11 raises effective capacity to 540 Mbps and drops utilisation to 57% — at no cost.

Code Example 3 — network workbench (HTML)

```
/* Access points sharing ONE channel must take turns with each other, so six units on
   one channel deliver roughly one unit's worth of airtime -- not six. The number of
   radios that genuinely add capacity is therefore capped by the number of distinct
   non-overlapping channels in use. This single line is the whole of Section 3.4. */
const effectiveRadios = Math.min(aps, channels);
const wirelessCapacity = effectiveRadios * apCapacity;

/* Compare the two utilisations and let the LARGER one name the constraint. */
const verdict = linkUtil > wirelessUtil
  ? 'The internet connection is the constraint. More bandwidth is justified.'
  : 'The wireless side is the constraint. More bandwidth would change nothing.';
```

- **Task:** three tools in one offline file — an encapsulation animator, a channel planner, and this bandwidth-versus-airtime assessor.
- **Try it:** set the link to 60 Mbps and the verdict flips. Then restore 500 and change channels from 1 to 3 — the wireless constraint disappears with no purchase.
- Opens in any browser, no internet needed. **399 lines** · ``code/L3_network_workbench.html``

Lesson 3 — video and checkpoint

- **Video (under 15 minutes):** “The Internet: Wires, Cables & Wi-Fi” — Code.org, YouTube.
 - <https://www.youtube.com/watch?v=ZhEf7e4kopM>
 - Code.org publishes this series for classroom use; it is still **linked, not copied**, under the terms in Section 12.4.
- **Checkpoint:**
 - 1. A user reports that one website is slow but everything else is fine. Which step of the eight-step trace does that point to?
 - 2. Why does adding a fourth access point on channel 6 make the problem worse?
 - 3. A video call stutters while a large download runs at full speed. Which measure is the problem, and which is fine?
 - 4. State the one reason more bandwidth would not have helped Ashworth Residence.

LESSON 4

Securing Information Systems

Sections 4.1 – 4.9 ·  01:44 → 02:00

What security protects, and how we describe risk

Property	The guarantee	A failure looks like
Confidentiality	Only authorised people can read the data	Student records emailed to the wrong list
Integrity	The data is correct and has not been altered undetectably	A grade changed with no record of who changed it
Availability	Authorised people can reach the data when needed	Ransomware; the registration system down on deadline day

- The CIA triad is a **checklist for describing damage precisely**. “We were hacked” is not an assessment. The three also trade against each other.
- **The vocabulary of risk:** **asset** (worth protecting) · **vulnerability** (a weakness) · **threat** (something that could exploit it) · **exploit** (the method) · **risk** (likelihood × impact) · **control** (reduces one or both).
- Precision here is what lets you **price** a risk in Section 4.5 instead of arguing about it.

The threats you will actually meet

Threat	How it works	Why it succeeds
Phishing / social engineering	A message persuades a person to act	It targets judgement, not software. No patch exists.
Malware & ransomware	Code that damages, steals or encrypts	One click, one unpatched machine, one shared drive
Credential theft	Reused or stolen passwords	One breach elsewhere unlocks your systems too
Insider threat	Authorised access, misused or mishandled	The controls were designed to let this person in
Unpatched software	A known flaw, publicly documented	The fix existed; nobody applied it

The first row causes the most incidents by a wide margin — and it is the only one you cannot buy your way out of.

Defence in depth: seven layers

- **1. Physical** — locks, badges, cameras. Nothing else matters if the machine can be carried out.
- **2. Perimeter** — firewalls, filtering at the boundary.
- **3. Network** — segmentation, so one compromised machine does not reach everything.
- **4. Endpoint** — patching, anti-malware, disk encryption.
- **5. Application** — input validation, authentication, secure development.
- **6. Data** — encryption at rest and in transit, access control, backup.
- **7. People and process** — training, verification procedures, incident plans.
- The principle: **assume every single layer will fail**, and arrange matters so that one failure is survivable.

The number that makes security a management decision

```
SLE  = Asset Value  x  Exposure Factor      # cost of ONE occurrence
ALE  = SLE  x  ARO      # expected cost PER YEAR
ROSI = (ALE_before - ALE_after - Cost) / Cost
```

- **SLE** (Single Loss Expectancy) — what one occurrence costs.
- **ARO** (Annualized Rate of Occurrence) — how many times per year you expect it. 0.5 means once every two years.
- **ALE** (Annualized Loss Expectancy) — the annual cost of living with the risk.
- **ROSI** (Return on Security Investment) — whether the control returns more than it costs. Negative means **do not buy it**.
- These estimates are uncertain, and that is fine. **The value is that the argument moves from opinion to arithmetic**, where the assumptions can be challenged one by one.

When it goes wrong: the six phases

Phase	What you do	The mistake people make
1. Preparation	Plan, train, and assign roles before anything happens	Writing the plan during the incident
2. Identification	Confirm it is real, and determine scope	Assuming scope instead of confirming it
3. Containment	Stop the spread, short-term then long-term	Wiping evidence while containing
4. Eradication	Remove the cause, not only the symptom	Restoring from an infected backup
5. Recovery	Restore service, monitor for recurrence	Declaring victory too early
6. Lessons learned	Change something so it cannot recur	Skipping this, guaranteeing a repeat

Aligned with the incident-handling model published by NIST (National Institute of Standards and Technology) — <https://www.nist.gov>

Real-life example: the bursar's office clicks a link

- **Task:** assess a real-shaped incident — classify the damage, identify the failed layer, price the risk, rank three proposals. **Relation:** Outcome 8.
- A message appearing to come from the Vice-Principal asks the bursar to update a supplier's banking details. The bursar complies. **CAD 41,500** leaves the account. It passed the spam filter; the domain differed by **one character**. Fictional.

Question	Finding
Confidentiality	Breached — supplier banking details and internal process were exposed
Integrity	Breached — a fraudulent payment instruction was accepted as genuine
Availability	Not affected — every system stayed up throughout
Which defence layer failed?	Layer 7: people and process. There was no verification step for changes to payment details.
Which layers held?	Perimeter, endpoint and application all worked exactly as designed

This is why the assessment matters. Buying a better email filter addresses a layer that did not fail. The missing control was a **telephone callback** — a process, costing nothing.

Pricing the risk

Risk	SLE	ARO	ALE	Rank
R1 Payment-redirectation fraud	CAD 41,500	0.87	CAD 36,000	1
R2 Ransomware on a shared drive	CAD 29,000	0.75	CAD 21,750	2
R3 Credential theft via reuse	CAD 11,000	0.86	CAD 9,500	3
R4 Student-record disclosure	CAD 22,000	0.40	CAD 8,800	4
R5 Lost unencrypted laptop	CAD 6,000	0.30	CAD 1,800	5
		Total	CAD 77,850	

The register is what turns “we should improve security” into a **ranked, priced list** — and it names R1 as the place to spend first.

Which control is worth buying?

Proposed control	Cost/yr	ALE after	Reduction	ROSI	Decision
Telephone callback for banking changes	CAD 0	CAD 7,200	CAD 28,800	infinite	IMPLEMENT — free
Multi-factor authentication for all staff	CAD 4,300	CAD 3,000	CAD 33,000	+667%	Justified
Advanced email filtering appliance	CAD 18,000	CAD 21,000	CAD 15,000	−17%	NOT justified
Quarterly external penetration testing	CAD 26,000	CAD 33,000	CAD 3,000	−88%	NOT justified

The most expensive proposal ranks last, and the free one ranks first. Without the arithmetic, the appliance would probably have been purchased — because it is the one that looks like security.

Code Example 4 — risk workbench (HTML)

```
// Each indicator carries a WEIGHT reflecting how strongly it predicts fraud, and a
// RATIONALE so the tool teaches the reasoning rather than just producing a number.
const INDICATORS = [
  { id: 'domain',    weight: 25, label: 'Sender domain differs from the real one',
    why: 'One-character substitutions survive a quick glance and every spam filter.' },
  { id: 'urgency',   weight: 15, label: 'Message applies time pressure',
    why: 'Urgency exists to prevent the verification step you are about to skip.' },
  { id: 'banking',   weight: 30, label: 'Requests a change to payment details',
    why: 'This is the single highest-value action an attacker can request.' },
  // ... five further indicators, 8 in total
];
```

- **Task:** score a suspicious message against eight weighted indicators, and rank security proposals by ROSI using the SLE/ARO/ALE calculator.
- Pre-loaded with the three Fairview proposals so you can reproduce the table on the previous slide, then change the assumptions and watch the ranking move.
- Fully offline. **309 lines** · ``code/L4_risk_workbench.html``

Lesson 4 — video and checkpoint

- **Video (under 15 minutes):** “Cybersecurity and Crime” — Code.org, YouTube.
 - https://www.youtube.com/watch?v=AuYNXgO_f3Y
 - Linked under the terms in Section 1.8 and Section 12.4. If a link has moved, the written summary in the lecture covers the same ground.
- **Checkpoint:**
 - 1. Classify the Fairview incident against the CIA triad, in three short sentences.
 - 2. A control costs CAD 12,000 and reduces an ALE of CAD 20,000 to CAD 6,000. Calculate ROSI and state the decision.
 - 3. Which layer of defence in depth failed at Fairview, and which control fixes it for nothing?
 - 4. Which of the six phases do organisations skip, and what does skipping it guarantee?

What the four lessons are really one lesson about

- Each lesson taught the same discipline applied to different equipment: **measure before you spend**.
- **Infrastructure** — the Print Hub did not buy a server, because a five-year model said renting was CAD 15,040 cheaper.
- **Databases** — the Robotics Club did not hire someone to clean the sheet, because keys and constraints made the mess impossible.
- **Networks** — Ashworth Residence did not buy bandwidth, because the link was at 61% and the constraint was airtime.
- **Security** — Fairview did not buy the CAD 18,000 appliance, because its ROSI was negative and a free process change scored better.
- **Assessment is what separates the answer that feels right from the one that is.**

Graded lab task: The Meridian Institute Assessment

- **Scenario:** a 900-student institute has budget for exactly one of four proposals. You must decide which, and defend it.
- **What you produce (100 marks):**
 - 1. Infrastructure — a five-year TCO comparison, using the Lesson 1 file (20)
 - 2. Data — normalise their spreadsheet into tables with keys (20)
 - 3. Network — a bandwidth-and-airtime budget with a verdict (20)
 - 4. Security — a risk register and ROSI ranking (25)
 - 5. Recommendation — one page, one decision, defended with your own numbers (15)
- **Every file you need is already downloaded.** The task reuses the four code files and four workbooks from this week — nothing new to install.

Your downloadable files, and where to verify everything

- **Files (nothing needs an internet connection to use):** 1 lecture document (~30,700 words) · 2 Python files · 2 self-contained HTML tools · 6 SVG diagrams · 4 XLSX workbooks with live formulas · 7 CSV reference tables · this deck · the interactive web module.
- **Standards and technical authorities**
 - IETF — <https://www.ietf.org> · IEEE — <https://www.ieee.org> · ISO — <https://www.iso.org> · W3C — <https://www.w3.org>
- **Security and privacy authorities**
 - NIST — <https://www.nist.gov> · CISA — <https://www.cisa.gov> · Canadian Centre for Cyber Security — <https://www.cyber.gc.ca>
 - ENISA — <https://www.enisa.europa.eu> · OWASP — <https://owasp.org>
- **Documentation you will actually use**
 - Python — <https://docs.python.org/3/> · SQLite — <https://www.sqlite.org/docs.html> · MDN — <https://developer.mozilla.org>

SVG (Scalable Vector Graphics) · XLSX (Excel Open XML Spreadsheet) · CSV (Comma-Separated Values). **Section 11 explains why each link is worth your time.**

Disclaimer and terms of use (1/2)

- **Ownership.** This material — text, code, diagrams, spreadsheets and slides — is the original work and intellectual property of the **Cyber.SoHo Educational Hub**. Provided for enrolled educational use only. No part may be reproduced, redistributed, resold or used to train a machine-learning model without prior written permission.
- **No affiliation.** No sponsorship, endorsement, affiliation or partnership with any company, product, standards body or institution named here is claimed or implied. All trademarks belong to their respective owners and are referenced for identification and educational commentary only.
- **Fictional scenarios.** Northside Campus Print Hub, the Robotics Club, Ashworth Residence, Fairview College and the Meridian Institute are **invented**. All figures, measurements and incidents are illustrative. Any resemblance to a real organisation or event is coincidental.

Disclaimer and terms of use (2/2)

- **External links.** Third-party sites are not controlled by Cyber.SoHo. Videos are **linked, never copied or re-hosted**, and remain the property of their publishers. Before opening any external link, verify it: <https://www.virustotal.com/gui/home/url>
- **Lawful and ethical use only.** The security content is defensive and assessment-oriented. Applying it to systems you do not own or lack written authorisation to test may be a criminal offence — Criminal Code of Canada s.342.1; US CFAA 18 U.S.C. §1030; EU Directive 2013/40/EU; Spain, Ley Orgánica 10/1995.
- **No warranty, no liability.** All code and files are provided “as is” without warranty of any kind. Run them in a safe environment. To the fullest extent permitted by law, Cyber.SoHo accepts no liability for any loss arising from use of this material. It is educational content, not professional legal, financial or security advice.



Measure before you spend.

Four lessons, one discipline.

Cyber.SoHo Educational Hub

From the Industry to the Classroom — Building the IT's Future, Today and Together!

© 2026 Cyber.SoHo Educational Hub · All rights reserved · Educational use only